PRAKTIKUM 6
CLASS

NAMA: Ihsanta Zaki Sanjaya

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Dr Bima Sena Bayu Dewantara S.ST, M.T

MATA KULIAH: Praktikum Pemograman Dasar 2

TANGGAL: Rabu, 24 April 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

Praktikum 4
Class

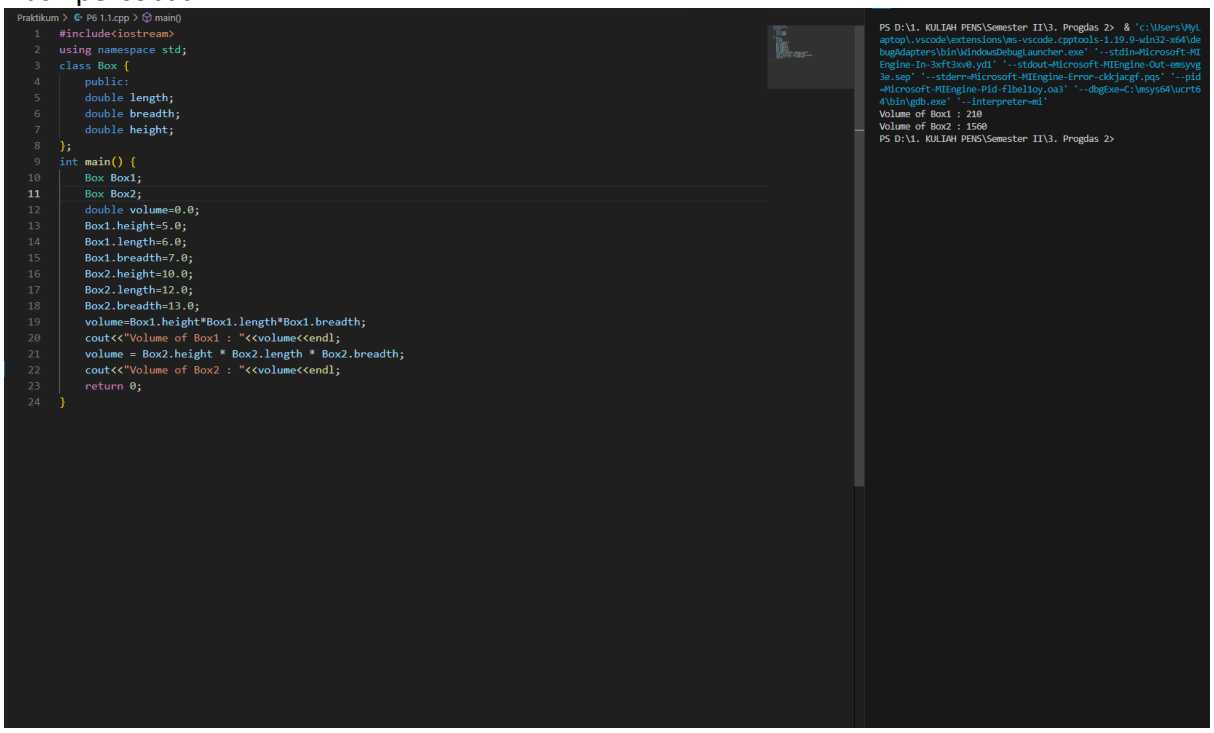
Untuk memberikan pemahaman kepada anda tentang konsep dasar class dan cara menggunakan dalam program C++, maka cobalah beberapa modul percobaan dibawah ini.

1. Percobaan 1 – Mengakses anggota class

```
#include <iostream>

using namespace std;

class Box
{
	public:
	double length;  // Length of a box
	double breadth;  // Breadth of a box
	double height;  // Height of a box
};

int main( )
{
	Box Box1;  // Declare Box1 of type Box
	Box Box2;  // Declare Box2 of type Box
	double volume = 0.0;  // Store the volume of a box here

	// box 1 specification
	Box1.height = 5.0;
	Box1.length = 6.0;
	Box1.breadth = 7.0;

	// box 2 specification
	Box2.height = 10.0;
	Box2.length = 12.0;
	Box2.breadth = 13.0;

	// volume of box 1
	volume = Box1.height * Box1.length * Box1.breadth;
	out << "Volume of Box1 : " << volume <<endl;

	// volume of box 2
	volume = Box2.height * Box2.length * Box2.breadth;
	cout << "Volume of Box2 : " << volume <<endl;
```

```
        return 0;
}
```

Hasil percobaan:



Output program:

```
Volume of Box1 : 210
Volume of Box2 : 1560
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

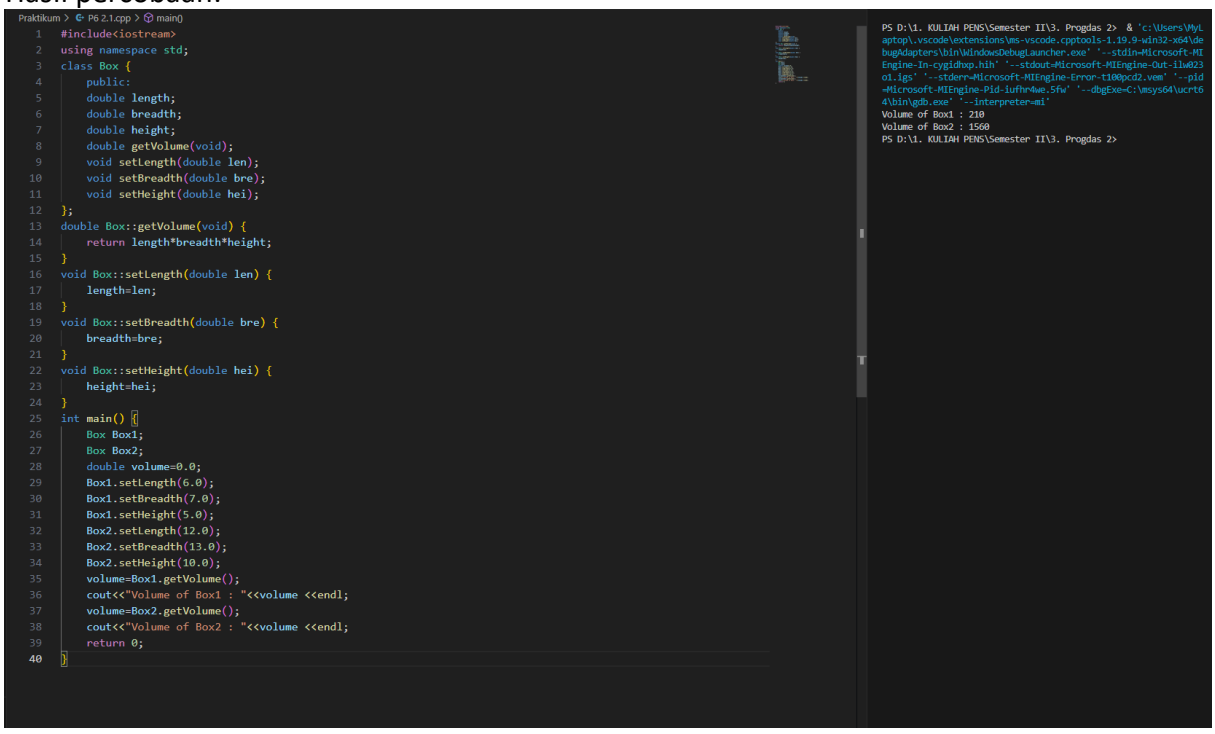
Dapat disimpulkan bahwa percobaan ini berfungsi untuk menghitung volume dua buah kotak menggunakan kelas `Box` dalam bahasa pemrograman C++. Kelas `Box` memiliki tiga atribut publik: `length` (panjang), `breadth` (lebar), dan `height` (tinggi). Pada fungsi `main`, dua objek `Box1` dan `Box2` dideklarasikan, masing-masing diinisialisasi dengan nilai atribut yang berbeda. Volume dari masing-masing kotak dihitung dengan mengalikan nilai `height`, `length`, dan `breadth` dari objek tersebut. Hasil perhitungan volume kemudian dicetak ke layar menggunakan `cout. Sehingga, percobaan ini menunjukkan cara mendefinisikan kelas, membuat objek, menetapkan nilai atribut, dan melakukan operasi perhitungan menggunakan objek-objek tersebut

2. Percobaan 2 – Fungsi sebagai anggota class

```
#include <iostream>

using namespace std;

class Box
{
	public:
	double length; // Length of a box
	double breadth; // Breadth of a box
	double height; // Height of a box

	// Member functions declaration
	double getVolume(void);
	void setLength(double len);
	void setBreadth(double bre);
	void setHeight(double hei);
};

// Member functions definitions
double Box::getVolume(void)
{
	return length * breadth * height;
}

void Box::setLength(double len)
{
	length = len;
}

void Box::setBreadth(double bre)
{
	breadth = bre;
}

void Box::setHeight(double hei)
{
	height = hei;
```

```
}

// Main function for the program
int main( )
{
        Box Box1; // Declare Box1 of type Box
        Box Box2; // Declare Box2 of type Box
        double volume = 0.0; // Store the volume of a box here

        // box 1 specification
        Box1.setLength(6.0);
        Box1.setBreadth(7.0);
        Box1.setHeight(5.0);

        // box 2 specification
        Box2.setLength(12.0);
        Box2.setBreadth(13.0);
        Box2.setHeight(10.0);

        // volume of box 1
        volume = Box1.getVolume();
        cout << "Volume of Box1 : " << volume <<endl;

        // volume of box 2
        volume = Box2.getVolume();
        cout << "Volume of Box2 : " << volume <<endl;

        return 0;
}
```

Hasil percobaan:



Output program:

```
Volume of Box1 : 210
Volume of Box2 : 1560
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini mengimplementasikan konsep dasar dari pemrograman berorientasi objek (OOP) menggunakan bahasa pemrograman C++. Kode ini mendefinisikan sebuah kelas `Box` yang memiliki tiga atribut publik: `length` (panjang), `breadth` (lebar), dan `height` (tinggi). Selain itu, kelas ini juga memiliki beberapa fungsi anggota untuk mengatur nilai atribut-atribut tersebut (`setLength`, `setBreadth`, dan `setHeight`) serta sebuah fungsi anggota untuk menghitung volume dari kotak (`getVolume`).

Dalam fungsi `main`, dua objek `Box1` dan `Box2` dari kelas `Box` dideklarasikan. Kemudian, atribut-atribut dari kedua objek tersebut diatur menggunakan fungsi-fungsi `setLength`, `setBreadth`, dan `setHeight`. Setelah itu, volume dari kedua kotak dihitung dengan memanggil fungsi `getVolume` pada masing-masing objek dan hasilnya dicetak ke layar menggunakan `cout`. Implementasi ini menunjukkan beberapa konsep OOP seperti enkapsulasi, di mana data dan fungsi yang beroperasi pada data tersebut dikelompokkan dalam satu unit (kelas), serta penggunaan fungsi anggota untuk mengakses dan memanipulasi data anggota kelas. Selain itu, kode ini juga mengilustrasikan cara mendefinisikan dan menggunakan fungsi anggota dalam sebuah kelas.

3. Percobaan 3 – Akses anggota class bertipe public

```
#include <iostream>

using namespace std;

class Line
{
	public:
	double length;

	void setLength(double len);
	double getLength(void);
};

// Member functions definitions
double Line:: getLength(void)
{
	return length ;
}

void Line:: setLength(double len)
{
	length = len;
}

// Main function for the program
int main()
{
	Line line;

	// set line length
	line.setLength(6.0);
	cout << "Length of line : " << line. getLength() <<endl;

	// set line length without member function
	line.length = 10.0; // OK: because length is public
	cout << "Length of line : " << line. length <<endl;

	return 0;
}
```

Hasil percoban:

```
Praktikum > P6 3.1.cpp > main()
#include<iostream>
using namespace std;
class Line {
    public:
    double length;
    void setLength(double len);
    double getLength(void);
};
double Line::getLength(void) {
    return length;
}
void Line::setLength(double len) {
    length=len;
}
int main() {
    Line line;
    line.setLength(9.0);
    cout<<"Length of line : "<<line. getLength()<<endl;
    line.length = 21.0;
    cout<<"Length of line : "<<line. length<<endl;
    return 0;
}
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-uvpzrbod.01m' '--stdout=Microsoft-MIEngine-Out-yqz0rawl.bul' '--stderr=Microsoft-MIEngine-Error-hr1idav4.j3s' '--pid=Microsoft-MIEngine-Pid-sllfpi4f.dyi' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Length of line : 9
Length of line : 21
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Output program:

```
Length of line : 9
Length of line : 21
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini menunjukkan dua cara mengakses dan mengubah atribut publik dari sebuah objek dalam pemrograman berorientasi objek (OOP) menggunakan C++. Pertama, dengan menggunakan fungsi anggota `setLength` dan `getLength` untuk mengatur dan mendapatkan nilai atribut `length`. Kedua, dengan mengakses atribut `length` secara langsung karena atribut ini dideklarasikan sebagai publik. Meskipun akses langsung lebih sederhana, ini mengabaikan prinsip enkapsulasi yang penting dalam OOP untuk menjaga integritas data. Oleh karena itu, praktik terbaik adalah menggunakan fungsi anggota untuk mengakses dan memodifikasi data, memastikan kontrol yang lebih baik terhadap perubahan yang dilakukan pada atribut.

4. Percobaan 4 – Akses anggota class bertipe private

```
#include <iostream>

using namespace std;

class Box
{
	public:
	double length;
	void setWidth(double wid);
	double getWidth(void);

	private:
	double width;
};

// Member functions definitions
double Box:: getWidth(void)
{
	return width ;
}

void Box:: setWidth(double wid)
{
	width = wid;
}

// Main function for the program
int main()
{
	Box box;

	// set box length without member function
	box.length = 10.0; // OK: because length is public
	cout << "Length of box : " << box.length <<endl;

	// set box width without member function
	// box.width = 10.0; // Error: because width is private
	box.setWidth(10.0);  // Use member function to set it.
	cout << "Width of box : " << box.getWidth() <<endl;

	return 0;
}
```

Hasil percobaan:

Praktikum > P6 4.1.cpp > main()

```cpp
#include<iostream>
using namespace std;
class Box {
    public:
    double length;
    void setWidth(double wid);
    double getWidth(void);
    private:
    double width;
};
double Box::getWidth(void) {
    return width;
}
void Box::setWidth(double wid) {
    width=wid;
}
int main() {
    Box box;
    box.length=10.0;
    cout<<"Length of box : "<<box.length <<endl;
    box.setWidth(10.0);
    cout<<"Width of box : "<<box.getWidth() <<endl;
    return 0;
}
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-0odeupy1.hiy' '--stdout=Microsoft-MIEngine-Out-mr2e5pe2.vdh' '--stderr=Microsoft-MIEngine-Error-sl33zpjs.eeo' '--pid=Microsoft-MIEngine-Pid-1qricr1n.xin' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Length of box : 10
Width of box : 10
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Output program:

```
Length of box : 10
Width of box : 10
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini menunjukkan perbedaan antara atribut publik dan privat dalam sebuah kelas di C++. Atribut `length` dideklarasikan sebagai publik, sehingga dapat diakses dan diubah langsung dari luar kelas. Sebaliknya, atribut `width` dideklarasikan sebagai privat, sehingga hanya bisa diakses dan diubah melalui fungsi anggota `setWidth` dan `getWidth`. Hal ini menegaskan pentingnya prinsip enkapsulasi dalam OOP, di mana atribut privat membantu menjaga integritas data dengan membatasi akses langsung dan mengontrol modifikasi melalui fungsi anggota.

5. Percobaan 5 – Akses anggota class bertipe protected

```
#include <iostream>

using namespace std;

class Box
{
	protected:
	double width;
};

class SmallBox:Box // SmallBox is the derived class.
{
	public:
	void setSmallWidth(double wid);
	double getSmallWidth(void);
};

// Member functions of child class
double SmallBox:: getSmallWidth(void)
{
	return width;
}

void SmallBox:: setSmallWidth(double wid)
{
	width = wid;
}

// Main function for the program
int main()
{
	SmallBox box;

	// set box width using member function
	box. setSmallWidth(5.0);
	cout << "Width of box : " << box.getSmallWidth() << endl;

	return 0;
}
```

Hasil percobaan:

Praktikum > P6 5.1.cpp > Box

```cpp
#include<iostream>
using namespace std;
class Box {
    protected:
    double width;
};
class SmallBox:Box {
    public:
    void setSmallWidth(double wid);
    double getSmallWidth(void);
};
double SmallBox::getSmallWidth(void) {
    return width;
}
void SmallBox:: setSmallWidth(double wid) {
    width = wid;
}
int main() {
    SmallBox box;
    box.setSmallWidth(5.0);
    cout<<"Width of box : "<<box.getSmallWidth()<<endl;
    return 0;
}
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-fcigo231.jig' '--stdout=Microsoft-MIEngine-Out-jvofisha.h23' '--stderr=Microsoft-MIEngine-Error-r0fcoh22.g1p' '--pid=Microsoft-MIEngine-Pid-kchhirae.fqs' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Width of box : 5
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Output program:

```
Width of box : 5
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini menunjukkan penggunaan pewarisan (inheritance) dalam pemrograman berorientasi objek (OOP) menggunakan C++. Kelas `SmallBox` adalah kelas turunan dari `Box` dan mewarisi atribut `width` yang bersifat `protected`, sehingga dapat diakses oleh kelas turunan. Fungsi anggota `setSmallWidth` dan `getSmallWidth` dalam kelas `SmallBox` digunakan untuk mengatur dan mendapatkan nilai `width`. Sehingga menegaskan bagaimana pewarisan memungkinkan akses dan modifikasi atribut yang dilindungi di kelas induk melalui fungsi anggota di kelas turunan, sekaligus tetap menjaga prinsip enkapsulasi.

6. Percobaan 6 – Class constructor

```
#include <iostream>

using namespace std;

class Line
{
	public:
	void setLength( double len );
	double getLength( void );
	Line(); // This is the constructor

	private:
	double length;
};

// Member functions definitions including constructor
Line:: Line(void)
{
	cout << "Object is being created" << endl;
}

void Line:: setLength( double len )
{
	length = len;
}

double Line:: getLength( void )
{
	return length;
}

// Main function for the program
int main( )
{
	Line line;

	// set line length
	line. setLength(6.0);
	cout << "Length of line : " << line. getLength() <<endl;

	return 0;
}
```

Hasil percobaan:

Praktikum > P6 6.1.cpp > main()

```cpp
#include<iostream>
using namespace std;
class Line {
    public:
    void setLength( double len );
    double getLength( void );
    Line();
    private:
    double length;
};
Line:: Line(void) {
    cout<<"Object is being created"<<endl;
}
void Line:: setLength( double len ) {
    length = len;
}
double Line:: getLength( void ) {
    return length;
}
int main( ) {
    Line line;
    line. setLength(6.0);
    cout << "Length of line : " << line. getLength() <<endl;
    return 0;
}
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\Users\MyL
aptop\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\de
bugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MI
Engine-In-2iueos4q.bde' '--stdout=Microsoft-MIEngine-Out-3lldfi
of.aq0' '--stderr=Microsoft-MIEngine-Error-ohlpmgrn.oiv' '--pid
=Microsoft-MIEngine-Pid-t0bewjk0.xd1' '--dbgExe=C:\msys64\ucrt6
4\bin\gdb.exe' '--interpreter=mi'
Object is being created
Length of line : 6
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Output program:

```
Object is being created
Length of line : 6
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini menunjukkan cara mendefinisikan dan menggunakan konstruktor dalam sebuah kelas di C++. Kelas `Line` memiliki konstruktor yang mencetak pesan saat objek dibuat, serta fungsi anggota `setLength` dan `getLength` untuk mengatur dan mendapatkan nilai atribut `length` yang bersifat privat. Dalam fungsi `main`, objek `line` dibuat, memicu pemanggilan konstruktor, dan panjang garis diatur serta dicetak menggunakan fungsi anggota. Itulah pentingnya konstruktor dalam menginisialisasi objek dan penggunaan fungsi anggota untuk mengelola atribut privat, menjaga prinsip enkapsulasi dalam OOP

Menurut hasil percobaan anda, kapankah sebuah constructor akan dipanggil?

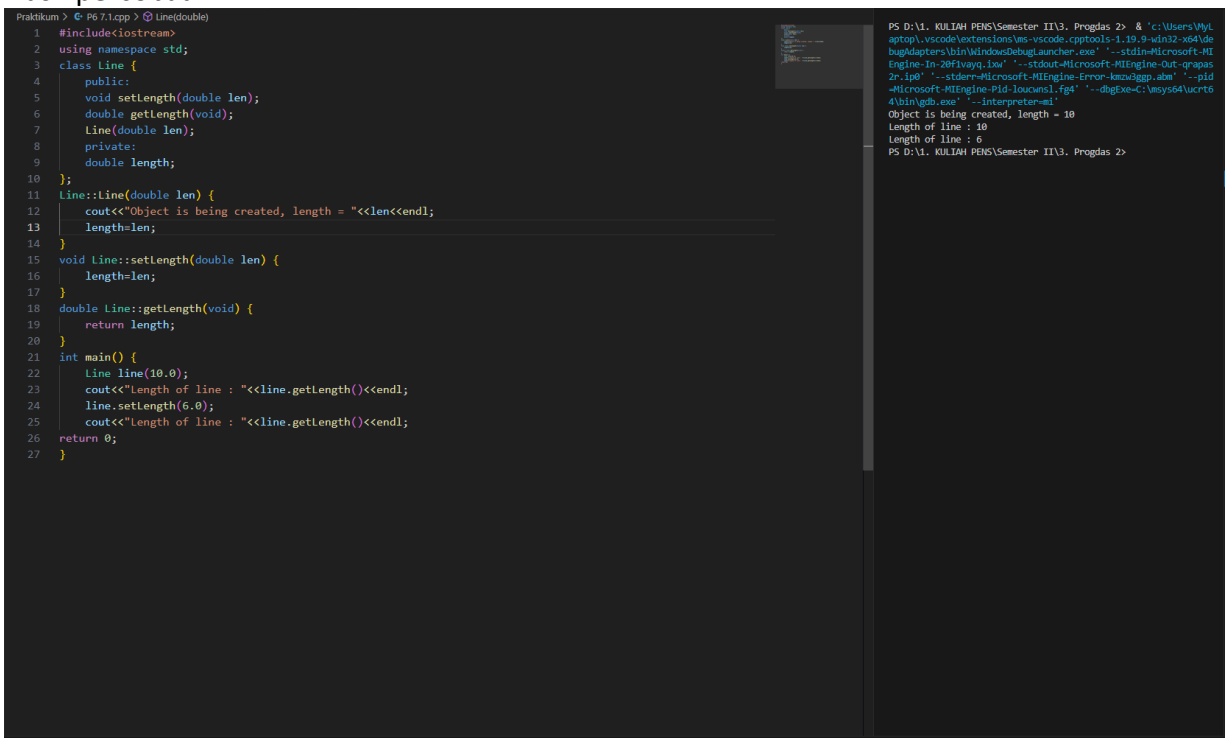
Berdasarkan output yang ada, konstruktor dipanggil pada saat objek `line` diciptakan dalam fungsi `main`. Jadi, konstruktor `Line::Line()` dipanggil ketika objek `line` dideklarasikan, sebelum pemanggilan fungsi `setLength(6.0)` dan `getLength()` yang menghasilkan output "Length of line : 6". Dengan demikian, konstruktor akan dipanggil saat objek dari kelas tersebut dibuat atau diinisialisasi.

7. Percobaan 7 – Class constructor dengan parameter

```
#include <iostream>

using namespace std;

class Line
{
	public:
	void setLength(double len);
	double getLength(void);
	Line(double len);  // This is the constructor

	private:
	double length;
};

// Member functions definitions including constructor
Line:: Line(double len)
{
	cout << "Object is being created, length = " << len << endl;
	length = len;
}

void Line:: setLength(double len)
{
	length = len;
}

double Line:: getLength(void)
{
	return length;
}

// Main function for the program
int main()
{
	Line line(10.0);

	// get initially set length.
	cout << "Length of line : " << line.getLength() <<endl;

	// set line length again
```

```
        line. setLength(6.0);
        cout << "Length of line : " << line.getLength() <<endl;

        return 0;
}
```

Hasil percobaan:



Output program:

```
Object is being created, length = 10
Length of line : 10
Length of line : 6
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini menunjukkan penggunaan konstruktor dalam sebuah kelas di C++. Konstruktor `Line::Line(double len)` dipanggil saat objek `line` dibuat dengan memberikan nilai awal pada saat inisialisasi. Pesan "Object is being created, length = [nilai len]" dicetak pada output untuk menandakan pemanggilan konstruktor. Selanjutnya, fungsi anggota `setLength` dan `getLength` digunakan untuk mengatur dan mendapatkan nilai atribut `length`. Percobaan ini menggambarkan penggunaan konstruktor untuk inisialisasi objek pada saat pembuatan objek serta penggunaan fungsi anggota untuk mengelola atribut objek setelah inisialisasi.

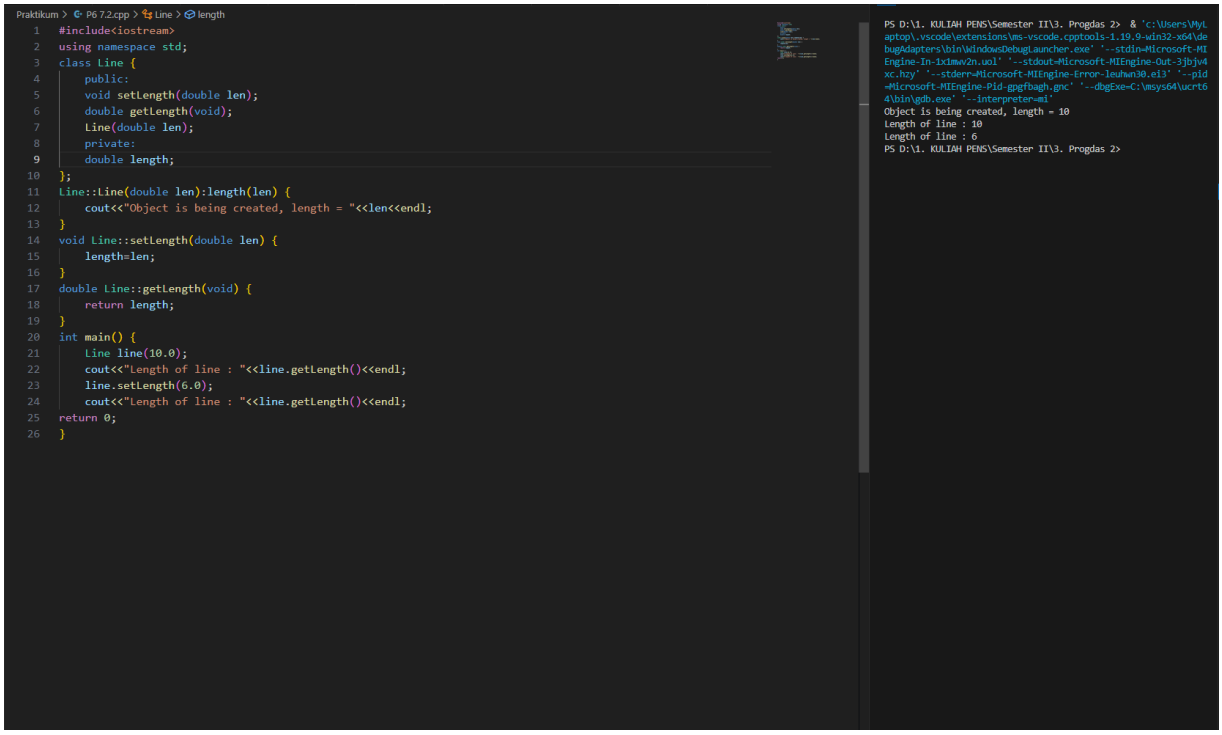
Gantilah bagian di bawah ini:

```
Line:: Line(double len)
{
        cout << "Object is being created, length = " << len << endl;
        length = len;
}
```

Dengan pengganti berikut ini:

```
Line::Line(double len): length(len)
{
        cout << "Object is being created, length = " << len << endl;
}
```

Hasil percobaan:



Output program:

```
Object is being created, length = 10
Length of line : 10
Length of line : 6
```

Berdasarkan hasil percobaan penggantian bagian tersebut, apakah yang bisa anda tarik sebagai kesimpulan? Jelaskan!

Dari hasil percobaan penggantian tersebut, kita dapat menarik kesimpulan bahwa penggunaan initialization list (`length(len)`) dalam konstruktor memberikan efisiensi dalam inisialisasi atribut. Dalam percobaan ini, konstruktor `Line::Line(double len): length(len)` secara langsung menginisialisasi atribut `length` dengan nilai `len` menggunakan initialization list, sehingga memungkinkan inisialisasi atribut secara langsung tanpa perlu menggunakan

pernyataan `length = len;` di dalam blok konstruktor. Hal ini dapat meningkatkan efisiensi kode dan menghasilkan kode yang lebih bersih dan ringkas. Kesimpulannya ialah penggunaan initialization list dalam konstruktor dapat meningkatkan efisiensi dan kejelasan dalam inisialisasi atribut.

8. Percobaan 8 – Class destructor

```
#include <iostream>

using namespace std;

class Line
{
	public:
	void setLength( double len );
	double getLength( void );
	Line();  // This is the constructor declaration
	~Line();  // This is the destructor: declaration

	private:
	double length;
};

// Member functions definitions including constructor
Line::Line(void)
```

```
{
	cout << "Object is being created" << endl;
}

Line::~Line(void)
{
	cout << "Object is being deleted" << endl;
}

void Line::setLength(double len)
{
	length = len;
}

double Line:: getLength(void)
{
	return length;
}

// Main function for the program
int main()
{
	Line line;

	// set line length
	line.setLength(6.0);
	cout << "Length of line : " << line.getLength() <<endl;

	return 0;
}
```

Hasil percobaan:

Praktikum > P7 8.1.cpp > main()

```cpp
#include<iostream>
using namespace std;
class Line {
    public:
    void setLength(double len);
    double getLength(void);
    Line();
    ~Line();
    private:
    double length;
};
Line::Line(void) {
    cout<<"Object is being created"<<endl;
}
Line::~Line(void) {
    cout<<"Object is being deleted"<<endl;
}
void Line::setLength(double len) {
    length=len;
}
double Line::getLength(void) {
    return length;
}
int main(){
    Line line;
    line.setLength(6.0);
    cout<<"Length of line :"<<line.getLength()<<endl;
    return 0;
}
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & `c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe` '--stdin=Microsoft-MIEngine-In-tknyaxsv.sgt' '--stdout=Microsoft-MIEngine-Out-llzpkeei.2gc' '--stderr=Microsoft-MIEngine-Error-wdrhf2a4.yic' '--pid=Microsoft-MIEngine-Pid-4rcxetwq.5ab' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Object is being created
Length of line :6
Object is being deleted
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Output program:

```
Object is being created
Length of line :6
Object is being deleted
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Menurut hasil percobaan anda, kapankah sebuah destructor akan dipanggil?

Dapat disimpulkan bahwa percobaan terdapat constructor yang digunakan untuk inisialisasi objek saat objek dibuat, sedangkan destructor dipanggil saat objek dihapus dari memori. Dalam kasus ini, destructor dipanggil setelah objek selesai digunakan, yaitu setelah fungsi `main()` selesai dieksekusi, menunjukkan bahwa objek `line` dihapus dari memori. Selain itu, terdapat juga penggunaan fungsi `setLength()` dan `getLength()` untuk mengatur dan mengambil panjang dari objek `line`.

9. Percobaan 9 – Copy constructor

```
#include <iostream>

using namespace std;

class Line
{
	public:
	int getLength(void);
	Line(int len); // simple constructor
	Line(const Line &obj); // copy constructor
	~Line(); // destructor

	private:
	int *ptr;
};

// Member functions definitions including constructor
Line::Line(int len)
{
	cout << "Normal constructor allocating ptr" << endl;
	// allocate memory for the pointer;
	ptr = new int;
	*ptr = len;
}

Line::Line(const Line &obj)
{
	cout << "Copy constructor allocating ptr." << endl;
	ptr = new int;
	*ptr = *obj.ptr; // copy the value
}

Line::~Line(void)
{
	cout << "Freeing memory!" << endl;
	delete ptr;
}

int Line::getLength(void)
{
	return *ptr;
}

void display(Line obj)
```

```
{
	cout << "Length of line : " << obj.getLength() <<endl;
}

// Main function for the program
int main()
{
	Line line(10);
	display(line) ;

	return 0;
}
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Percobaan ini menerapkan penggunaan constructor, copy constructor, dan destructor dalam sebuah class di C++. Pada percobaan tersebut, terdapat dua jenis constructor yang digunakan: constructor biasa (`Line(int len)`) dan copy constructor (`Line(const Line &obj)`). Constructor biasa digunakan untuk menginisialisasi objek dengan nilai yang diberikan saat objek dibuat, sedangkan copy constructor digunakan untuk membuat salinan dari objek yang ada.

```
#include <iostream>

using namespace std;

class Line
{
	public:
	int getLength(void);
	Line(int len); // simple constructor
	Line(const Line &obj); // copy constructor
	~Line(); // destructor

	private:
	int *ptr;
};

// Member functions definitions including constructor
Line::Line(int len)
{
	cout << "Normal constructor allocating ptr" << endl;

	// allocate memory for the pointer;
	ptr = new int;
	*ptr = len;
}

Line::Line(const Line &obj)
{
	cout << "Copy constructor allocating ptr." << endl;
```

```
	ptr = new int;
	*ptr = *obj.ptr; // copy the value
}

Line::~Line(void)
{
	cout << "Freeing memory!" << endl;
	delete ptr;
}

int Line::getLength(void)
{
	return *ptr;
}

void display(Line obj)
{
	cout << "Length of line : " << obj.getLength() <<endl;
}

// Main function for the program
int main()
{
	Line line1(10);

	Line line2 = line1; // This also calls copy constructor

	display(line1);
	display(line2);

	return 0;
}
```

Hasil percobaan:

Praktikum > P7 9.2.cpp > Line(int)

```cpp
#include <iostream>
using namespace std;
class Line {
    public:
    int getLength(void);
    Line(int len);
    Line(const Line &obj);
    ~Line();
    private:
    int *ptr;
};

Line::Line(int len) {
    cout << "Normal constructor allocating ptr" << endl;
    ptr = new int;
    *ptr = len;
}

Line::Line(const Line &obj) {
    cout << "Copy constructor allocating ptr." << endl;
    ptr = new int;
    *ptr = *obj.ptr;
}

Line::~Line(void) {
    cout << "Freeing memory!" << endl;
    delete ptr;
}

int Line::getLength(void) {
    return *ptr;
}

void display(Line obj) {
    cout << "Length of line : " << obj.getLength() << endl;
}

int main() {
    Line line1(10);
    Line line2 = line1;
    display(line1);
    display(line2);
    return 0;
}
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-xnfvpjsv.qv0' '--stdout=Microsoft-MIEngine-Out-euw40nut.5f2' '--stderr=Microsoft-MIEngine-Error-nsnbepmx.tov' '--pid=Microsoft-MIEngine-Pid-2f2hzqnj.x3r' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Normal constructor allocating ptr
Copy constructor allocating ptr.
Copy constructor allocating ptr.
Length of line : 10
Freeing memory!
Copy constructor allocating ptr.
Length of line : 10
Freeing memory!
Freeing memory!
Freeing memory!
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Output program:

```
Normal constructor allocating ptr
Copy constructor allocating ptr.
Copy constructor allocating ptr.
Length of line : 10
Freeing memory!
Copy constructor allocating ptr.
Length of line : 10
Freeing memory!
Freeing memory!
Freeing memory!
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Percobaan ini adalah pengembangan dari kode sebelumnya yang menambahkan penggunaan copy constructor dalam pembuatan objek `line2` dari objek `line1`. Perbedaan utama dengan kode sebelumnya adalah penambahan inisialisasi objek `line2` menggunakan copy constructor, yang membuat salinan dari objek `line1`.

10. Percobaan 10 – Friend functions

```
#include <iostream>

using namespace std;

class Box
{
	double width;

	public:
	friend void printWidth(Box box);
	void setWidth(double wid);
};

// Member function definition
void Box::setWidth(double wid)
{
	width = wid;
}

// Note: printWidth() is not a member function of any class.
void printWidth(Box box)
{
	/* Because printWidth() is a friend of Box, it can
	directly access any member of this class */

	cout << "Width of box : " << box.width <<endl;
}

// Main function for the program
int main()
{
	Box box;

	// set box width without member function
	box.setWidth(10.0);

	// Use friend function to print the wdith.
	printWidth(box);

	return 0;
}
```

Hasil percobaan:

```
Praktikum > P7 10.1.cpp > main()
#include<iostream>
using namespace std;
class Box {
    double width;
    public:
    friend void printWidth(Box box);
    void setWidth(double wid);
};
void Box::setWidth(double wid) {
    width=wid;
}
void printWidth(Box box) {
    cout<<"Width of box: "<<box.width<<endl;
}
int main (){
    Box box;
    box.setWidth(10.0);
    printWidth(box);
    return 0;
}
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-rxdfr5p0.1eg' '--stdout=Microsoft-MIEngine-Out-mbirl3nq.w5o' '--stderr=Microsoft-MIEngine-Error-fpuakvda.dii' '--pid=Microsoft-MIEngine-Pid-ddrv5tzn.f4f' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Width of box: 10
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Output program:

```
Width of box: 10
```

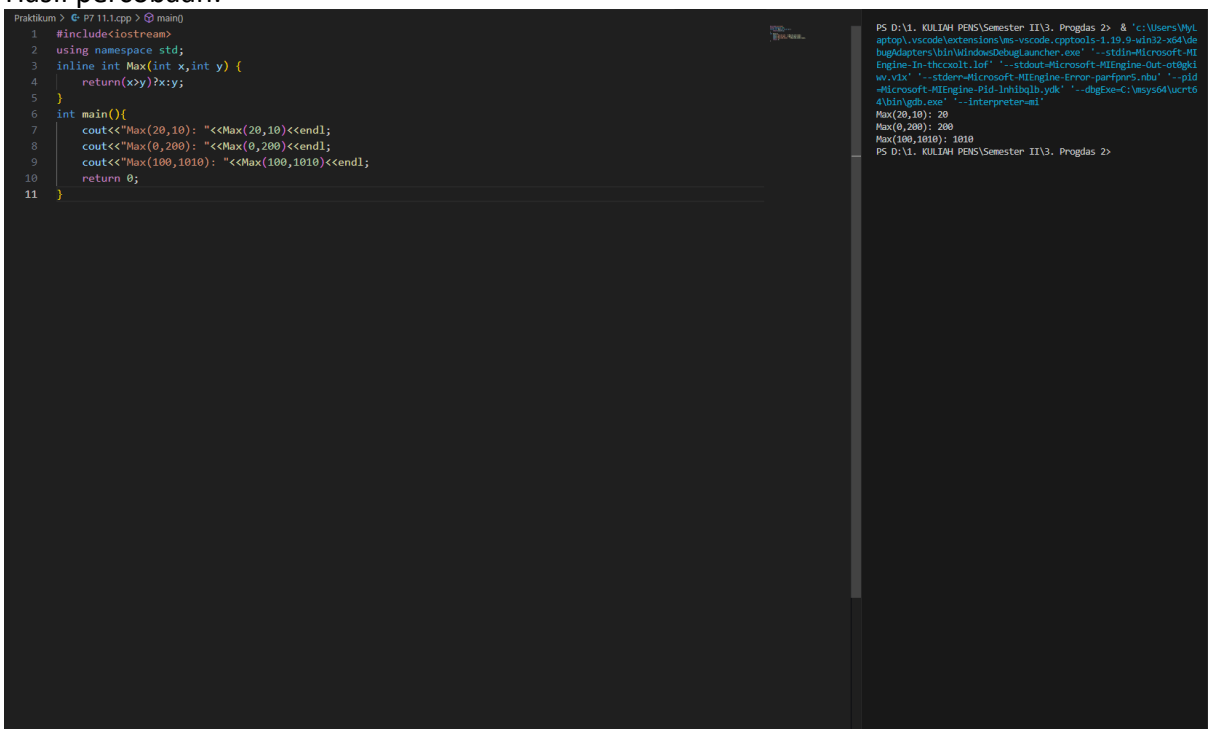
Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini mengilustrasikan penggunaan friend function. Friend function adalah fungsi yang dideklarasikan di luar class, namun memiliki akses langsung ke anggota-anggota private atau protected dari class tersebut. Pada contoh ini, fungsi `printWidth()` adalah friend function dari class `Box`, sehingga dapat mengakses member `width` yang private dari class `Box` tanpa melalui fungsi member. Dalam percobaan tersebut, `printWidth()` digunakan untuk mencetak nilai `width` dari objek `box` tanpa perlu menggunakan fungsi member `setWidth()`. Yang mana menunjukkan adanya fleksibilitas friend function dalam mengakses data private dari suatu class, meskipun tetap harus berhati-hati agar tidak mengurangi prinsip encapsulation dari OOP.

11. Percobaan 11 – Inline functions

```
#include <iostream>

using namespace std;

inline int Max(int x, int y)
{
        return (x > y)? x : y;
}

// Main function for the program
int main()
{
        cout << "Max (20,10): " << Max(20,10) << endl;
        cout << "Max (0,200): " << Max(0,200) << endl;
        cout << "Max (100,1010): " << Max(100,1010) << endl;

        return 0;
}
```

Hasil percobaan:



Output program:

```
Max(20,10): 20
Max(0,200): 200
Max(100,1010): 1010
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini mengilustrasikan penggunaan fungsi inline untuk mencari nilai maksimum antara dua bilangan. Fungsi inline digunakan untuk menghindari overhead yang terjadi saat pemanggilan fungsi, dengan langsung memasukkan kode fungsi ke dalam tempat pemanggilan. Dalam contoh ini, fungsi Max digunakan untuk membandingkan dua bilangan dan mengembalikan nilai maksimum di antara keduanya. Hal

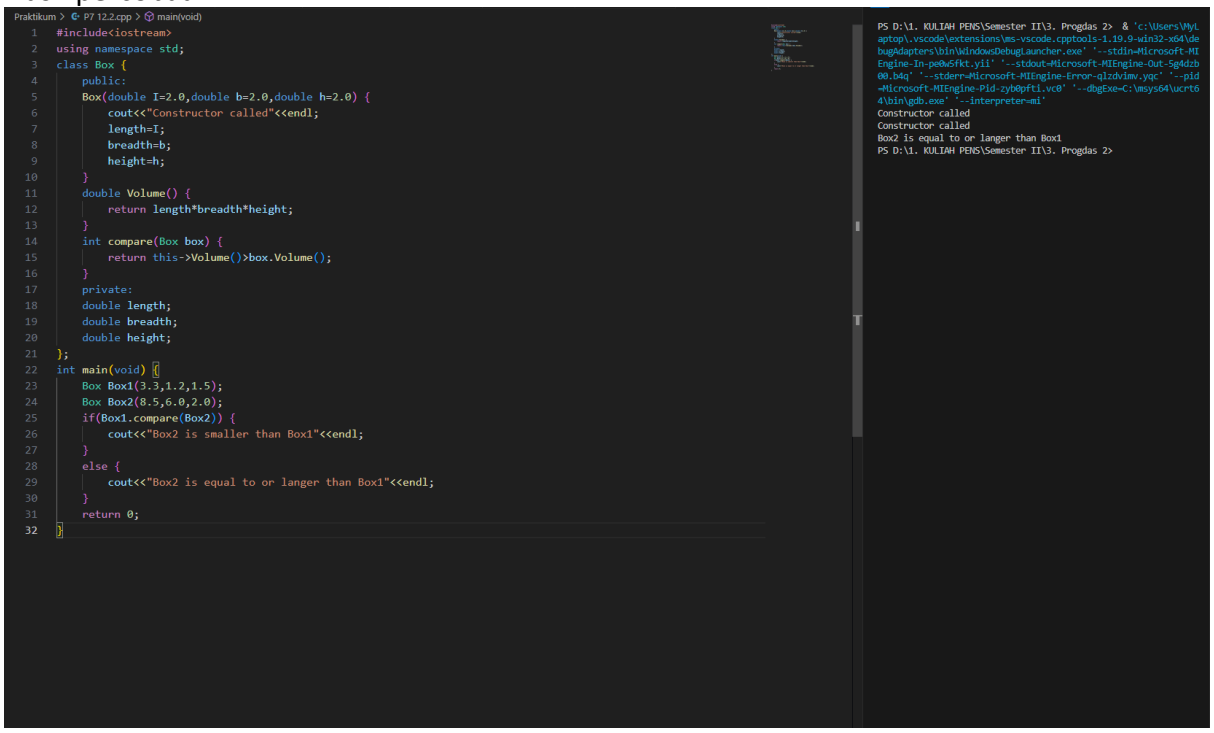
ini dapat mempercepat eksekusi program dengan mengurangi waktu yang dibutuhkan untuk pemanggilan fungsi.

12. Percobaan 12 – this pointer

```
#include <iostream>

using namespace std;

class Box
{
	public:
	// Constructor definition
	Box(double l=2.0,double b=2.0,double h=2.0)
	{
		cout <<"Constructor called." << endl;
		length = l;
		breadth = b;
		height = h;
	}

	double Volume()
	{
		return length * breadth * height;
	}
```

```
        int compare(Box box)
        {
                return this->Volume() > box.Volume();
        }

        private:
        double length; // Length of a box
        double breadth; // Breadth of a box
        double height; // Height of a box
};

int main(void)
{
        Box Box1(3.3, 1.2, 1.5); // Declare box1
        Box Box2(8.5, 6.0, 2.0); // Declare box2
        if(Box1.compare(Box2))
        {
                cout << "Box2 is smaller than Box1" <<endl;
        }
        else
        {
                cout << "Box2 is equal to or larger than Box1" <<endl;
        }

        return 0;
}
```

Hasil percobaan:



Output program:

```
Constructor called
Constructor called
Box2 is equal to or langer than Box1
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini menunjukkan cara mendefinisikan sebuah kelas dengan menggunakan konstruktor yang memiliki parameter default, serta metode untuk menghitung volume dan membandingkan dua objek dari kelas tersebut. Dengan mendemonstrasikan pembuatan dua objek `Box` dengan dimensi yang berbeda dan menggunakan metode `compare` untuk menentukan apakah satu kotak lebih besar dari yang lain berdasarkan volume. Hasil percobaan ini memperlihatkan bahwa `Box1` dengan volume lebih kecil dibandingkan `Box2`, sehingga program akan mencetak "Box2 is equal to or larger than Box1". Selain itu, percobaan ini juga menyoroti penggunaan `this` pointer untuk mengakses anggota kelas dalam metode.

13. Percobaan 13 – Pointer to C++ class

```
#include <iostream>

using namespace std;

class Box
{
	public:
	// Constructor definition
	Box(double l=2.0,double b=2.0,double h=2.0)
	{
		cout <<"Constructor called." << endl;
		length = l;
		breadth = b;
		height = h;
	}

	double Volume()
	{
		return length * breadth * height;
	}

	private:
	double length; // Length of a box
	double breadth; // Breadth of a box
	double height;  // Height of a box
};

int main(void)
{
	Box Box1(3.3, 1.2, 1.5); // Declare box1
	Box Box2(8.5, 6.0, 2.0); // Declare box2

	Box *ptrBox; // Declare pointer to a class.

	// Save the address of first object
	ptrBox = &Box1;
	// Now try to access a member using member access operator
	cout << "Volume of Box1: " << ptrBox->Volume() << endl;

	// Save the address of first object
	ptrBox = &Box2;
	// Now try to access a member using member access operator
	cout << "Volume of Box2: " << ptrBox->Volume() << endl;
```

```
        return 0;
}
```

Hasil percobaan:

Praktikum > P7 13.1.cpp > main(void)

```cpp
#include<iostream>
using namespace std;
class Box {
    public:
    Box(double l=2.0,double b=2.0,double h=2.0) {
        cout<<"Constructor called"<<endl;
        length=l;
        breadth=b;
        height=h;
    }
    double Volume() {
        return length*breadth*height;
    }
    private:
    double length;
    double breadth;
    double height;
};
int main(void) {
    Box Box1(3.3,1.2,1.5);
    Box Box2(8.5,6.0,2.0);
    Box *ptrBox;
    ptrBox=&Box1;
    cout<<"Volume of Box1: "<<ptrBox->Volume()<<endl;
    ptrBox=&Box2;
    cout<<"Volume of Box2: "<<ptrBox->Volume()<<endl;
    return 0;
}
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-cgwnefcv.1u3' '--stdout=Microsoft-MIEngine-Out-j4mxrxbx.drs' '--stderr=Microsoft-MIEngine-Error-dipdxmm4.53o' '--pid=Microsoft-MIEngine-Pid-vkqgtzzq.pvy' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Constructor called
Constructor called
Volume of Box1: 5.94
Volume of Box2: 102
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Output program:

```
Constructor called
Constructor called
Volume of Box1: 5.94
Volume of Box2: 102
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

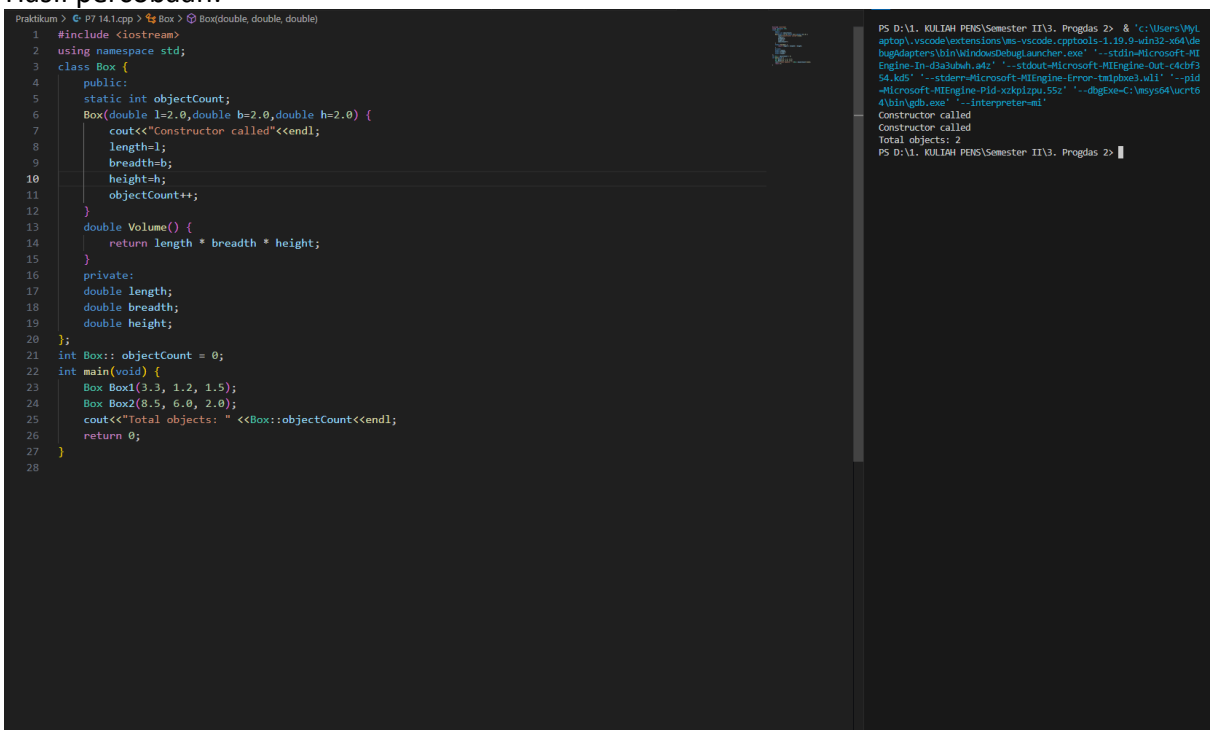
Dapat disimpulkan bahwa percobaan ini mendemonstrasikan pembuatan dua objek `Box` dengan dimensi yang berbeda dan penggunaan pointer untuk menyimpan alamat dari objek-objek ini. Pointer tersebut kemudian digunakan untuk memanggil metode `Volume` dan menghitung volume dari masing-masing objek `Box`. Hasil percobaan ini memperlihatkan bagaimana pointer dapat digunakan untuk mengakses metode dari objek yang ditunjuk, mencetak volume dari `Box1` dan `Box2`.

14. Percobaan 14 – Anggota statik dari sebuah class

```
#include <iostream>

using namespace std;

class Box
{
	public:
	static int objectCount;
	// Constructor definition
	Box(double l=2.0, double b=2.0, double h=2.0)
	{
		cout <<"Constructor called." << endl;
		length = l;
		breadth = b;
		height = h;
		// Increase every time object is created
		objectCount++;
	}

	double Volume()
	{
		return length * breadth * height;
	}

	private:
	double length;  // Length of a box
	double breadth;  // Breadth of a box
	double height;  // Height of a box
};

// Initialize static member of class Box
int Box:: objectCount = 0;

int main(void)
{
	Box Box1(3.3, 1.2, 1.5);  // Declare box1
	Box Box2(8.5, 6.0, 2.0);  // Declare box2
```

```
        // Print total number of objects.
        cout << "Total objects: " << Box::objectCount << endl;

        return 0;
}
```

Hasil percobaan:



Output program:

```
Constructor called
Constructor called
Total objects: 2
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

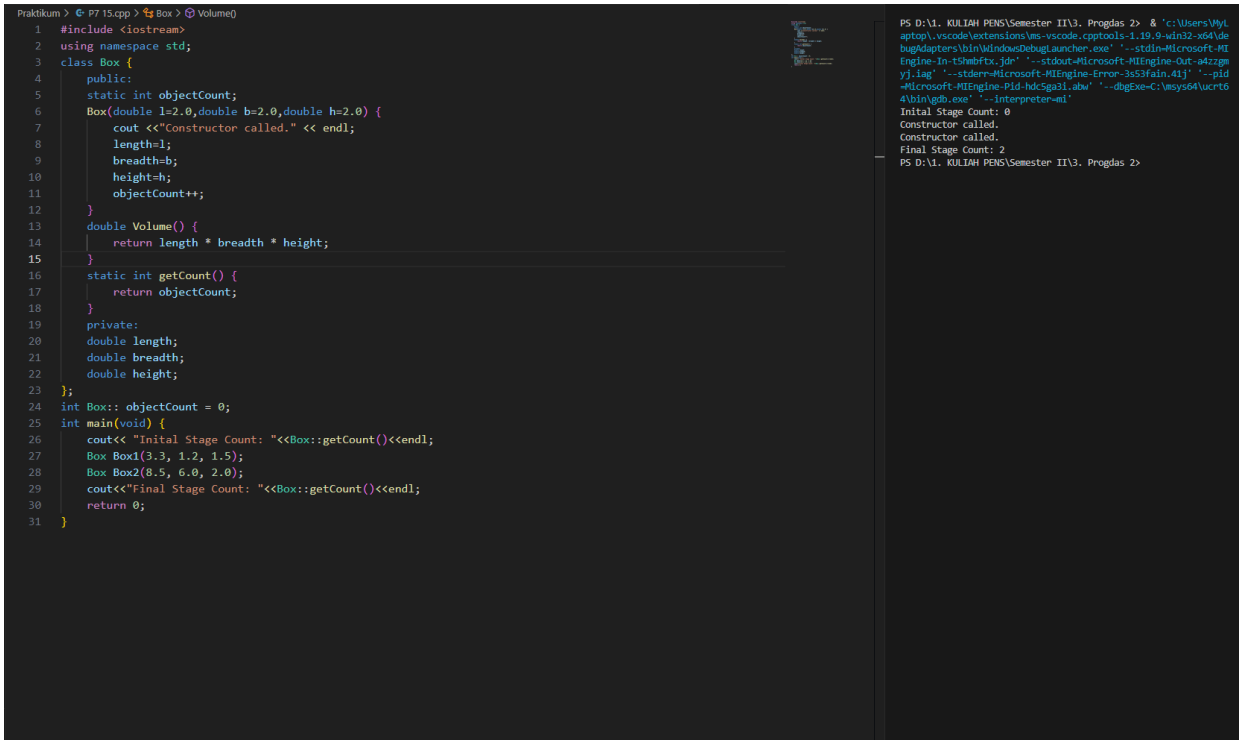
Dapat disimpulkan bahwa percobaan ini menunjukkan cara menggunakan anggota statis dalam sebuah kelas di C++ untuk melacak jumlah objek yang telah dibuat dari kelas tersebut. Dalam kode ini, kelas `Box` memiliki anggota statis `objectCount` yang diinisialisasi ke nol. Setiap kali sebuah objek `Box` dibuat, konstruktor akan dipanggil dan `objectCount` akan ditingkatkan. Kode ini mendemonstrasikan pembuatan dua objek `Box` dengan dimensi yang berbeda dan kemudian mencetak jumlah total objek `Box` yang telah dibuat. Hasil percobaan ini menunjukkan bahwa `objectCount` berhasil mencatat jumlah objek yang dibuat, yang ditunjukkan dengan output "Total objects: 2".

15. Percobaan 15 – Anggota fungsi statik

```
#include <iostream>

using namespace std;

class Box
{
	public:
	static int objectCount;
	// Constructor definition
	Box(double l=2.0, double b=2.0, double h=2.0)
	{
		cout <<"Constructor called." << endl;
		length = l;
		breadth = b;
		height = h;
		// Increase every time object is created
		objectCount++;
	}

	double Volume()
	{
		return length * breadth * height;
	}

	static int getCount()
	{
		return objectCount;
	}

	private:
	double length;  // Length of a box
	double breadth;  // Breadth of a box
	double height;  // Height of a box
};

// Initialize static member of class Box
int Box:: objectCount = 0;

int main(void)
{
	// Print total number of objects before creating object.
```

```
        cout << "Inital Stage Count: " << Box::getCount() << endl;

        Box Box1(3.3, 1.2, 1.5);  // Declare box1
        Box Box2(8.5, 6.0, 2.0);  // Declare box2

        // Print total number of objects after creating object.
        cout << "Final Stage Count: " << Box::getCount() << endl;

        return 0;
}
```

Hasil percobaan:



Output program:

```
Inital Stage Count: 0
Constructor called.
Constructor called.
Final Stage Count: 2
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Dapat disimpulkan bahwa percobaan ini menunjukkan cara menggunakan anggota statis dalam sebuah kelas di C++ untuk melacak dan mengakses jumlah objek yang telah dibuat dari kelas tersebut melalui metode statis. Dalam kode ini, kelas `Box` memiliki anggota statis `objectCount` yang diinisialisasi ke nol dan ditingkatkan setiap kali sebuah objek `Box` dibuat melalui konstruktor. Metode statis `getCount` digunakan untuk mengakses nilai `objectCount`. Kode ini mencetak jumlah total objek sebelum dan sesudah pembuatan dua objek `Box`, yang menunjukkan bahwa `objectCount` berhasil mencatat jumlah objek yang dibuat, dengan output yang menunjukkan "Initial Stage Count: 0" dan "Final Stage Count: 2".